PYTHON · ASSIGNMENT

Strings, Dictionaries, Sorting & JSON-style Data

10 questions based on the class notebook — attempt all questions.

Note — **Instructions:** Use only the concepts covered in class — string methods, f-strings, dictionary methods, `sorted()` with `key=lambda`, and loops. Write clean code and show the output of every program.

Part A · Strings (Q1 - Q4)

Q1. A user types their city carelessly: `"   nEw   dELhi   "`. Clean it and print exactly `New Delhi`. Also validate that the city contains only alphabets and spaces — print `Invalid city` otherwise.

OUTPUT

```
New Delhi
```

Q2. Prices arrive as one messy string: `" 120, 45 , 300 ,80, 95 "`. Convert it into a list of integers, then print the total bill, the highest price, and the average price with 2 decimal places (use an f-string with `:.2f`).

OUTPUT

```
Total   : 640
Highest : 300
Average : 128.00
```

Q3. Validate a username entered by the user (e.g. `"  Rahul_123 "`). Rules:

- clean the spaces and convert to lowercase
- must be alphanumeric (letters and digits only — no symbols)
- must **start with an alphabet**
- length must be between 5 and 12 characters

Print `Valid username` or `Invalid username`. (`"Rahul_123"` is invalid because of the underscore; `"rahul123"` is valid.)

Q4. For a given sentence, print: (a) the total number of words, (b) how many words **end** with a vowel, and (c) the longest word in the sentence.

Part B · Dictionaries (Q5 - Q8)

Q5. Count the frequency of each character in a string, **ignoring spaces** and treating uppercase and lowercase as the same letter. Example: `"Aba c"` → `{'a': 2, 'b': 1, 'c': 1}`.

Q6. Count the frequency of each word in a sentence and print the word that occurs the **most** number of times (use `sorted()` with `key=lambda` on the items, or a loop).

Q7. Invert the dictionary below so that values become keys. If two days share the same value, store both days in a **list**:

```
sales = {"mon": 200, "tue": 350, "wed": 200}
```

OUTPUT

```
{200: ['mon', 'wed'], 350: ['tue']}
```

Q8. Marks of two tests are stored in two dictionaries. Merge them into one dictionary — if a student appears in both, **sum** the marks:

```
test1 = {"amit": 40, "bina": 35}
test2 = {"amit": 30, "chirag": 45}
```

OUTPUT

```
{'amit': 70, 'bina': 35, 'chirag': 45}
```

Part C · Sorting + JSON-style Data (Q9 - Q10)

Q9. Sort the employees below by salary (highest first) using `sorted()` with `key=lambda`, and print each line using an f-string in the format shown:

```
employees = [("Ravi", 55000), ("Sita", 72000), ("Amit", 48000)]
```

OUTPUT

```
Sita earns Rs. 72000
Ravi earns Rs. 55000
Amit earns Rs. 48000
```

Q10. Using the list of dictionaries below:

```
movies = [
    {"title": "Movie A", "rating": 8.2, "genre": "Thriller"},
    {"title": "Movie B", "rating": 8.4, "genre": "Sports"},
    {"title": "Movie C", "rating": 8.1, "genre": "Comedy"},
    {"title": "Movie D", "rating": 8.2, "genre": "Drama"},
]
```

- (a) Print the titles of all movies with rating **> 8.1**.
- (b) Extract all genres into a list (no duplicates).
- (c) Build a dict → `{rating: [titles]}` grouping movies by rating, then print it rating by rating using f-strings, formatted nicely.